OPERATING SYSTEMS

Memory Management

Pavan A C

Department of Computer Science & Engineering

OPERATING SYSTEMS

Slides Credits for all the PPTs of this course



- The slides/diagrams in this course are an **adaptation**, **combination**, and **enhancement** of material from the following resources and persons:

1. Slides of Operating System Concepts, Abraham Silberschatz, Peter Baer Galvin, Greg Gagne - 9th edition 2013 and some slides from 10th edition 2018
2. Some conceptual text and diagram from Operating Systems - Internals and Design Principles, William Stallings, 9th edition 2018
3. Some presentation transcripts from A. Frank – P. Weisberg
4. Some conceptual text from Operating Systems: Three Easy Pieces, Remzi Arpaci-Dusseau, Andrea Arpaci Dusseau

OPERATING SYSTEMS

Structure of the page table

Pavan A C
Department of Computer Science & Engineering

Hardware Support for Page Table

- The **hardware implementation of page table can be done by using dedicated registers.**
- But the usage of **register** for the **page table** is satisfactory only if page table is **small**.
- If page table contain **large number of entries** then we can use **TLB(translation Look-aside buffer),** a special, small, fast look up **hardware cache.**

PES UNIVERSITY
CELEBRATING 50 YEARS

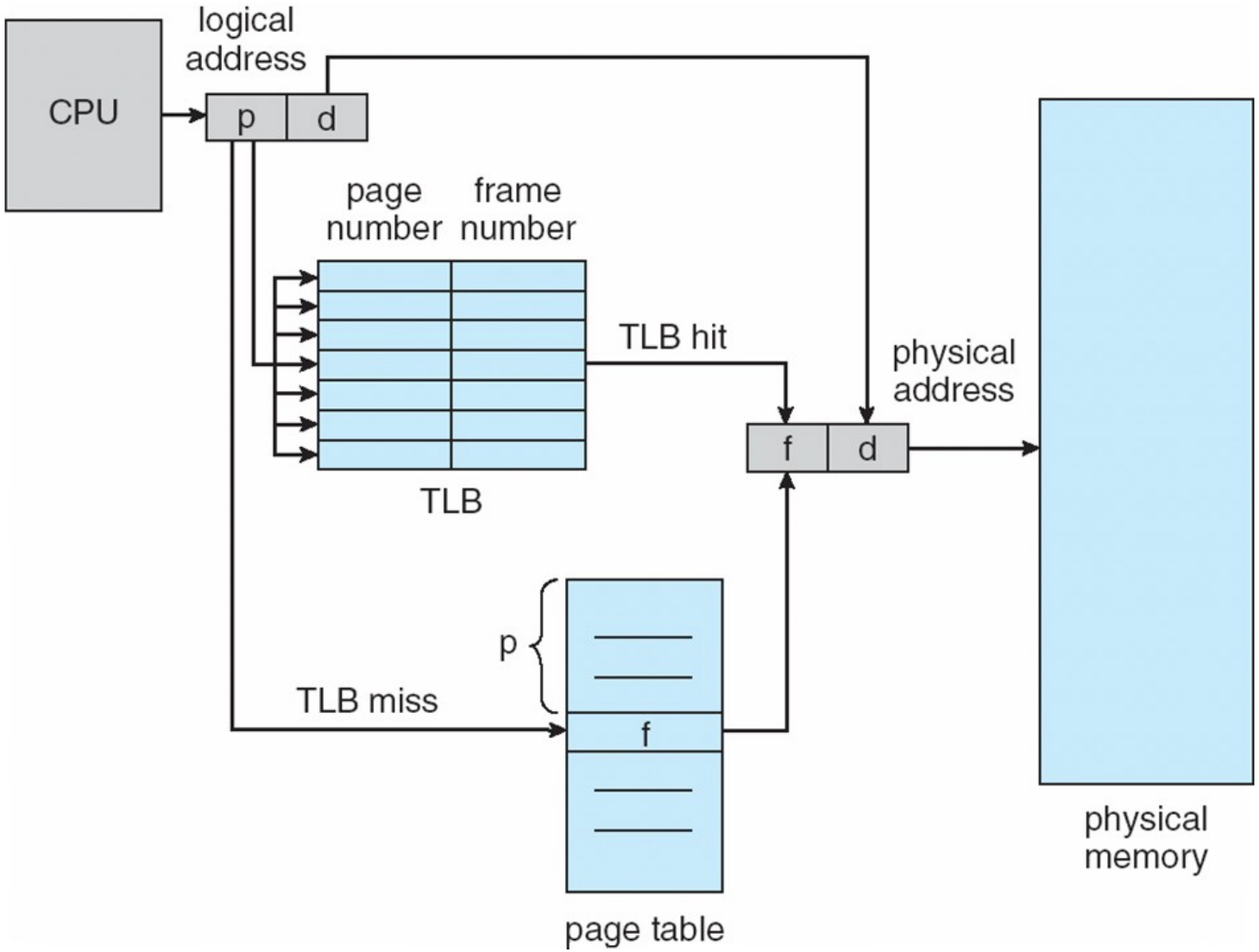
Hardware Support for Page Table



- Page table is kept in main memory
- **Page-table base register** (**PTBR**) points to the page table
- **Page-table length register** (**PTLR**) indicates size of the page table
- In this scheme every data/instruction **access requires two memory accesses**
  - **One for the page table and one for the data / instruction**
- The two memory access problem can be solved by the use of a special fast-lookup hardware cache called **associative memory** or **translation look-aside buffers** (**TLBs**)

OPERATING SYSTEMS

Paging Hardware With TLB



OPERATING SYSTEMS

Effective Access Time

- Hit ratio – **percentage of times that a page number is found in the TLB**
- An 80% hit ratio means that **we find the desired page number in the TLB 80% of the time**.
- Suppose that 10 nanoseconds to access memory.
  - If we find the desired page in TLB then a mapped-memory **access take 10 ns**
  - Otherwise **we need two memory access so it is 20 ns** i.e. if we fail to find the page number in the TLB then **we must first access memory for the page table and frame number (10 ns) and then access the desired byte in memory (10 nanoseconds)**, for a total of 20 ns (assuming that a page table lookup takes only one memory access).
- **Effective Access Time** (**EAT**)

  EAT = 0.80 x 10 + 0.20 x 20 = 12 nanoseconds

  implying 20% slowdown in access time
- Consider a more realistic hit ratio of 99%,

  EAT = 0.99 x 10 + 0.01 x 20 = 10.1ns

  implying only 1% slowdown in access time.

PES UNIVERSITY
CELEBRATING 50 YEARS

Translation Look-Aside Buffer



How TLB is different from CPU Cache

- TLB is about '**speeding up address translation for Virtual**/logical memory' so that page-table needn't to be accessed for every address.
- CPU Cache is about '**speeding up main memory** access latency' so that RAM isn't accessed always by CPU.
- TLB operation **comes at the time of address translation** by MMU while **CPU cache operation comes at the time of memory access** by CPU.

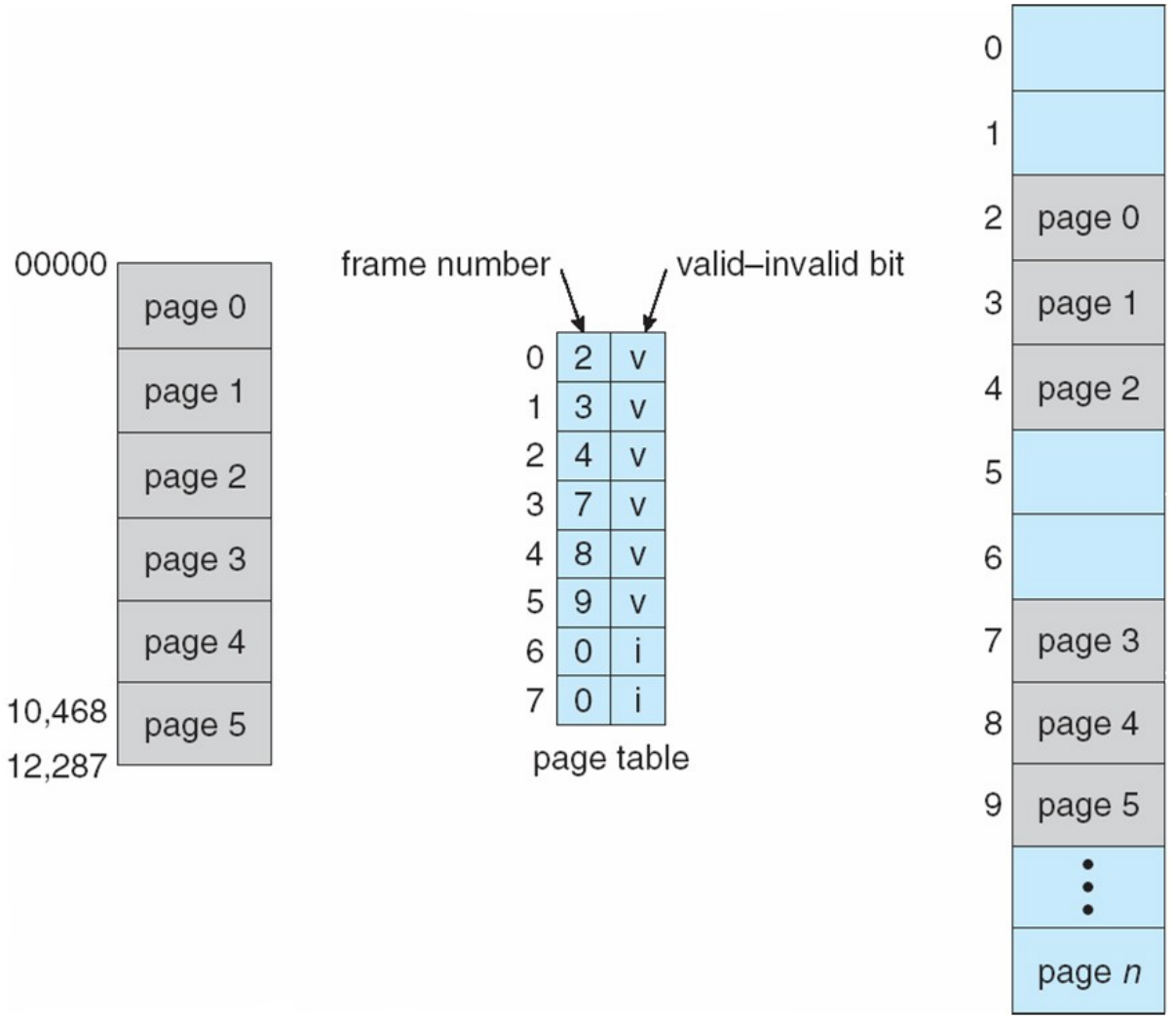
Memory Protection



- Memory protection implemented **by associating protection bit with each frame to indicate if read-only or read-write access is allowed**
  - Can also add more **bits to indicate page execute-only, and so on**
- **Valid-invalid** bit attached to each entry in the page table:
  - "valid" **indicates that the associated page is in the process' logical address space, and is thus a legal page**
  - "invalid" indicates that the page is not in the process' logical address space
  - Or use **page-table length register** (**PTLR**)
- Any violations result in a trap to the kernel

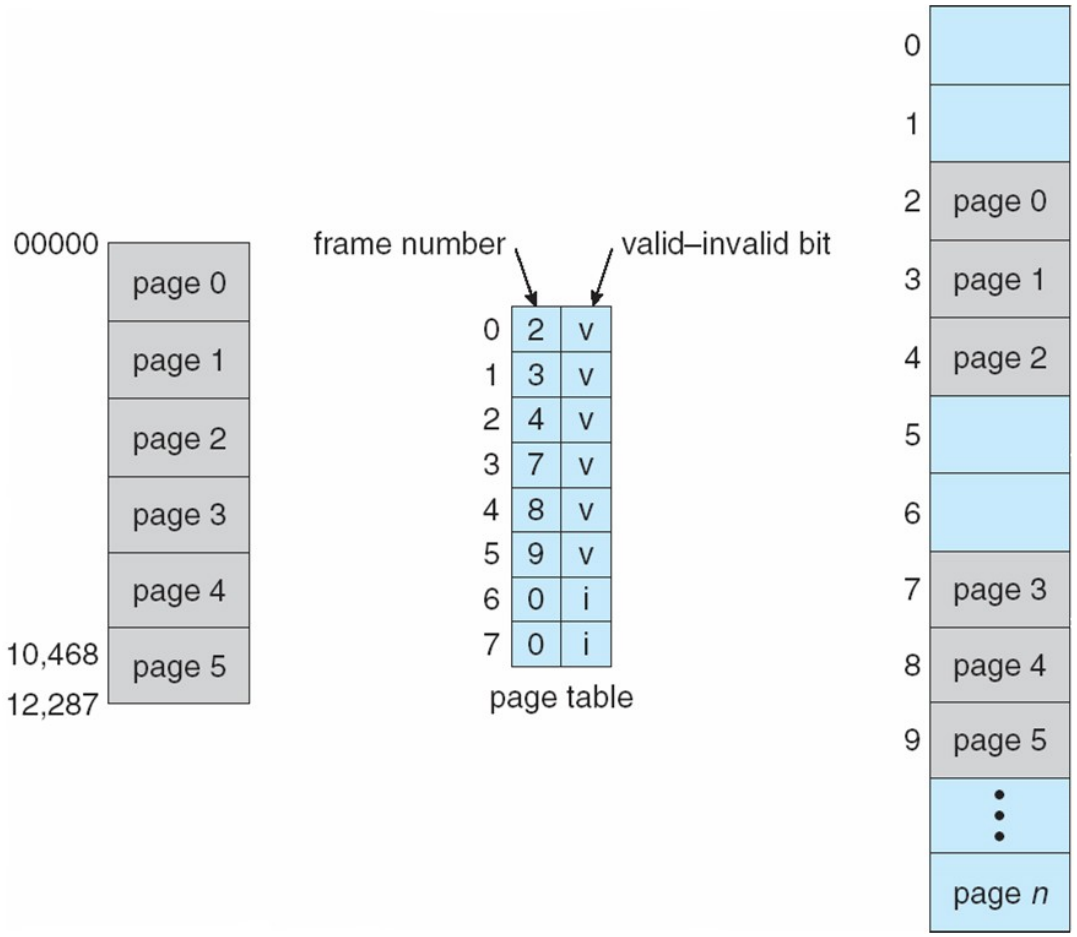
OPERATING SYSTEMS

Valid (v) or Invalid (i) Bit In A Page Table



OPERATING SYSTEMS

Valid (v) or Invalid (i) Bit In A Page Table



Demand Paging: If a page is marked "Invalid," **the MMU throws a Page Fault trap.** Modern OSes use this intentionally; they map memory as **invalid so they don't have to allocate physical RAM until the exact time the program tries to use it.**

OPERATING SYSTEMS

Shared Pages

- An advantage of paging **is the possibility of sharing common code**.
  - important in a time-sharing environment
- Consider a system that supports 40 users, each of whom executes a text editor.
- If the text editor consists of **150 KB of code and 50 KB of data space.**
- we need **8,000 KB(150 KBX40+50KB X40) to support the 40 users**.
- Can this memory space usage be reduced?
  - 4 YES
- If the code is shared, then a total space required will be **2,150 KB**

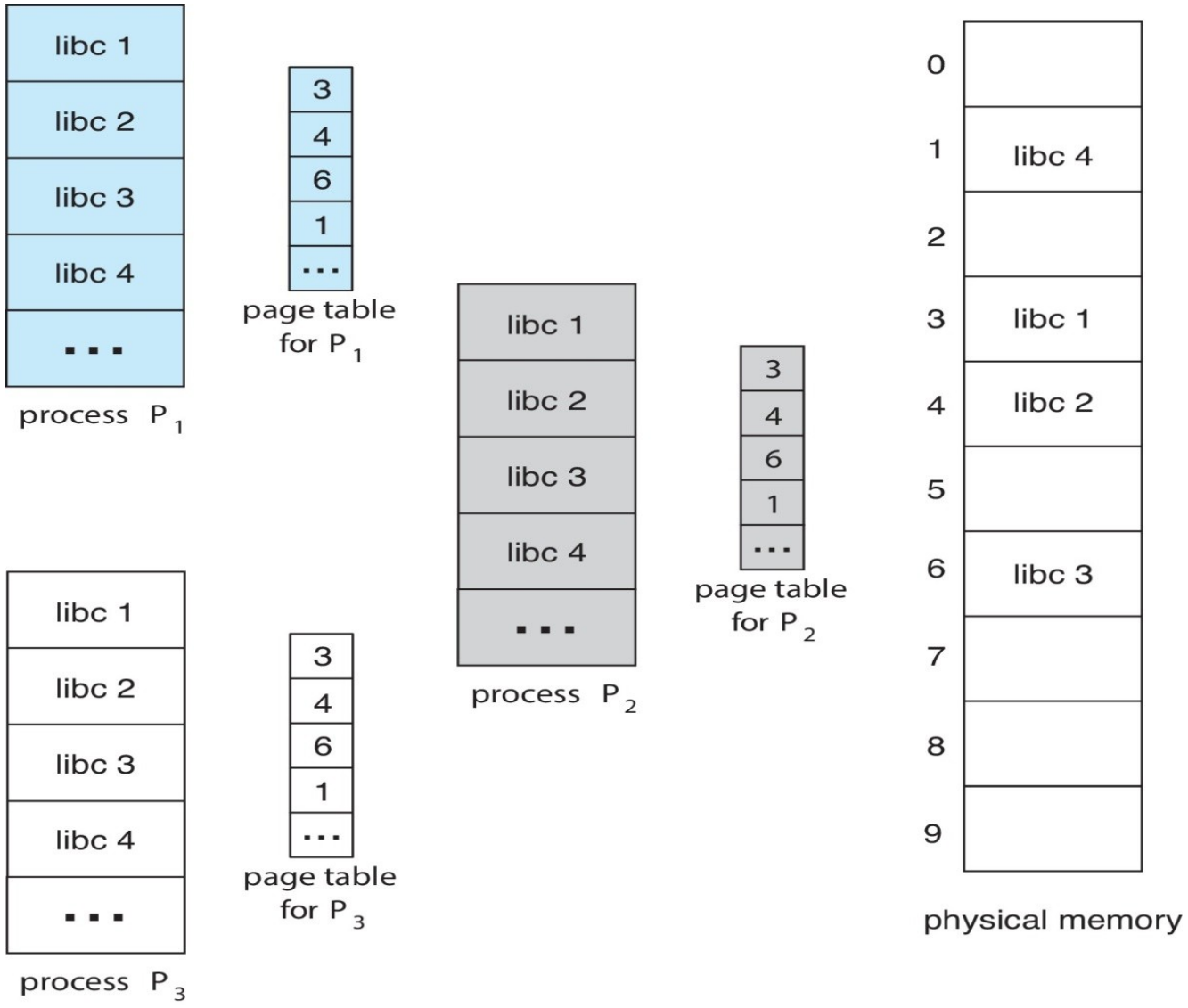
Shared Pages



- **Shared code**
  - One copy of read-only (**reentrant**) code shared among processes (i.e., text editors, compilers, window systems)
  - **Reentrant (multi-instance) code is a reusable routine** that multiple programs can invoke, interrupt, and reinvoke simultaneously.
  - Reentrant code **is non-self-modifying code**: it never changes during execution.
  - Similar to **multiple threads sharing the same process** space
  - Also **useful for interprocess communication** if sharing of read-write pages is allowed
- **Private code and data**
  - Each process keeps a separate copy of the **code and data**
  - The pages for the **private code and data can appear anywhere in the logical address space**

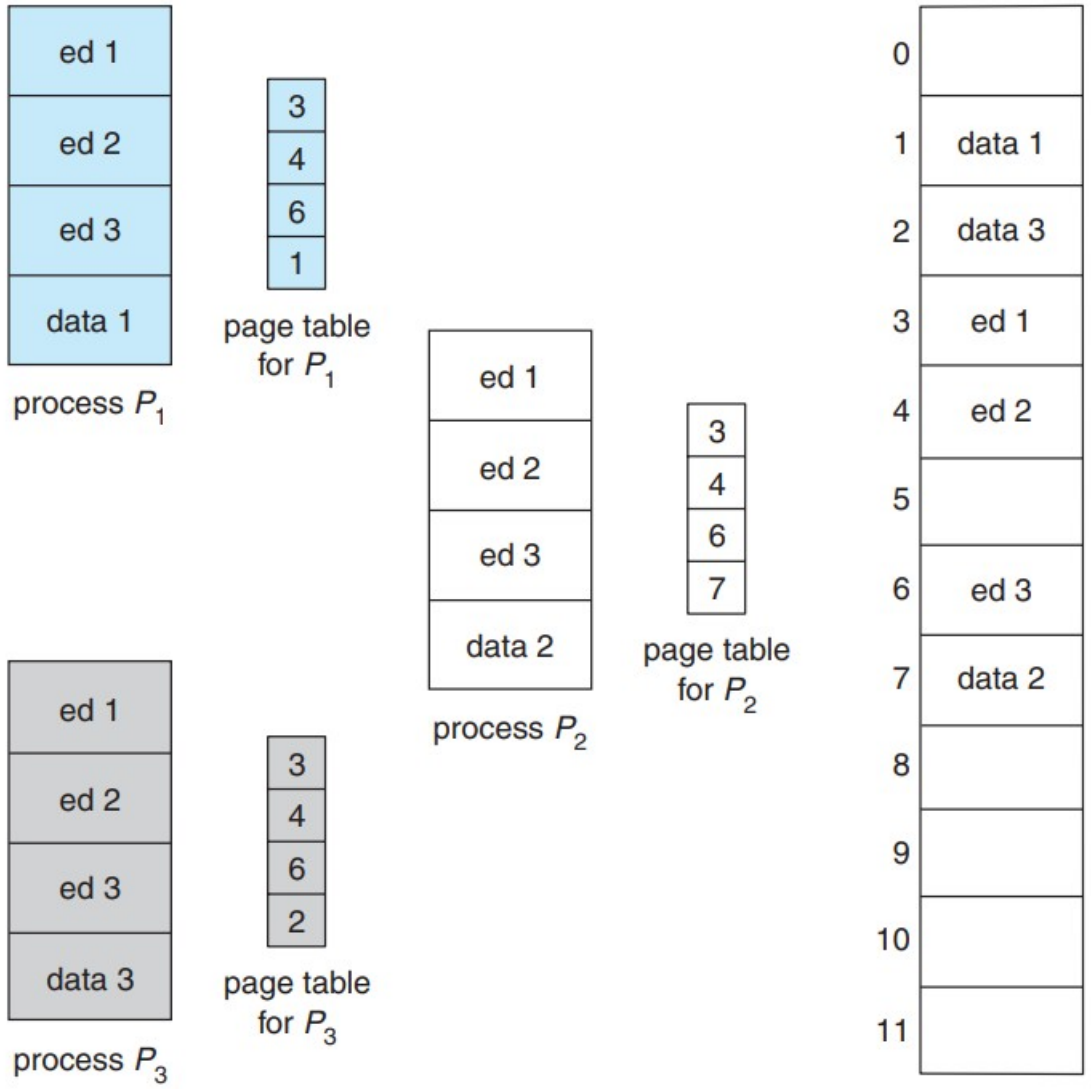
OPERATING SYSTEMS

Shared Pages Example



OPERATING SYSTEMS

Shared Pages Example



Structure of the Page Table

- Memory structures for paging can get huge using straight-forward methods
  - Consider a 32-bit logical address space as on modern computers
  - Page size of 4 KB (2^{12})
  - Page table would have 1 million entries (2^{32} / 2^{12})
  - If each entry is 4 bytes -> **4 MB of physical memory for page table alone**
    - **That amount of memory used to cost a lot**
    - Don't want to allocate that contiguously in main memory
- we explore some of the most common techniques for structuring the page table, including **hierarchical paging, hashed page tables, and inverted page tables**

OPERATING SYSTEMS

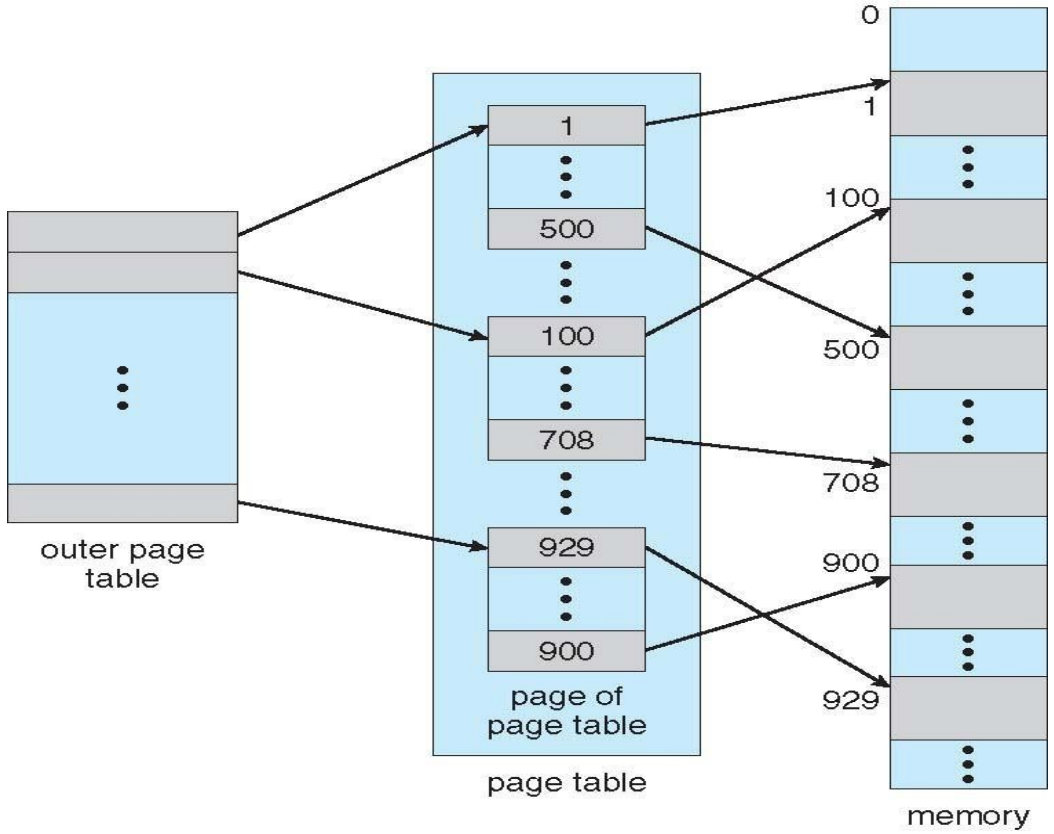
Structures of the Page Table

- Hierarchical Paging
- Hashed Page Tables
- Inverted Page Tables

PES UNIVERSITY
CELEBRATING 50 YEARS

Hierarchical Page Tables

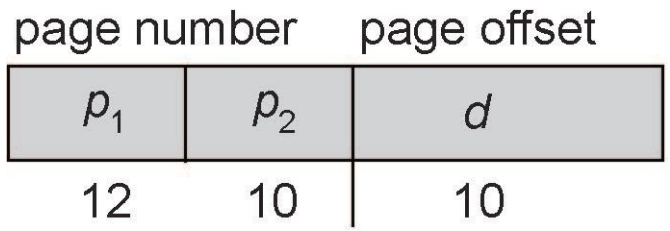
- Break up the logical address space into multiple page tables
- A simple technique is a two-level page table
- We then page the page table



OPERATING SYSTEMS

Two-Level Paging Example

- A logical address (on 32-bit machine with 1K page size) is divided into:
  - a page number consisting of 22 bits
  - a page offset consisting of 10 bits
- Since the page table is paged, the page number is further divided into:
  - a 12-bit page number
  - a 10-bit page offset
- Thus, a logical address is as follows:

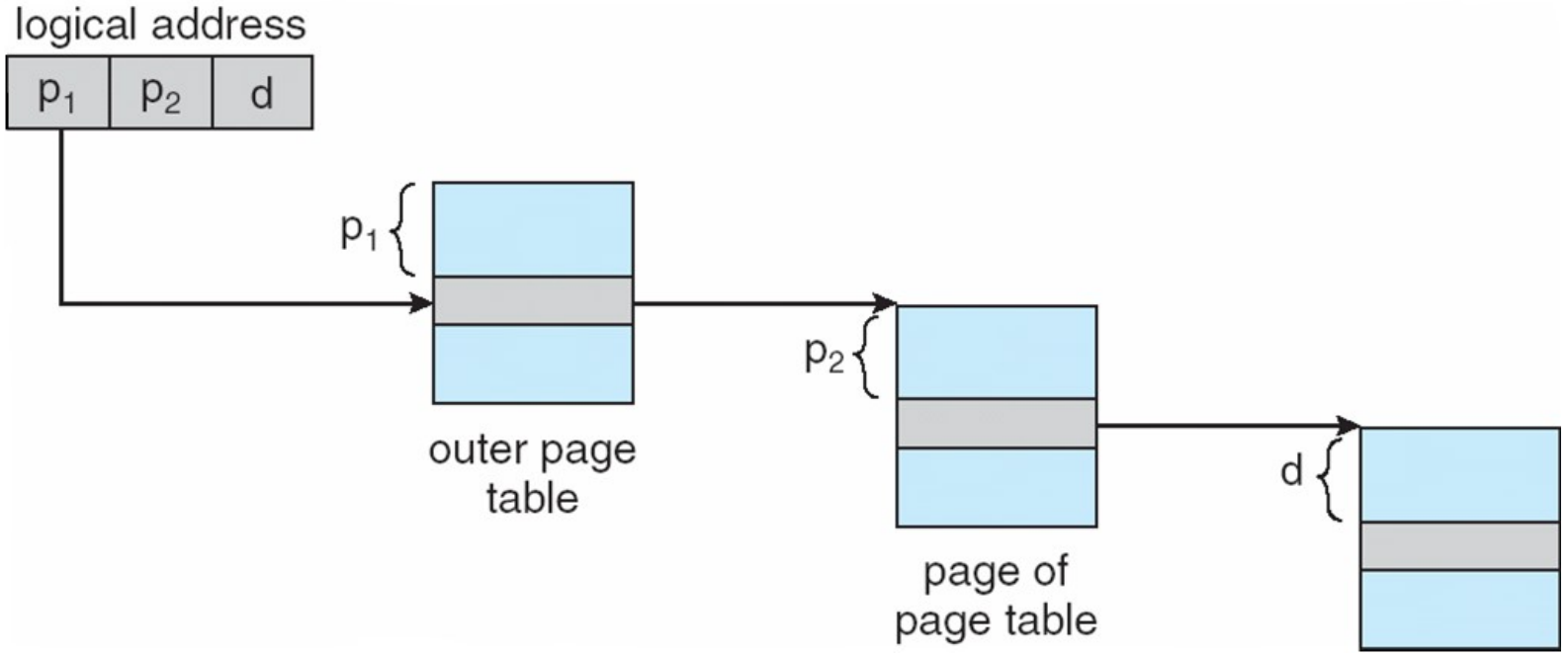


where p_1 is an index into the outer page table, and p_2 is the displacement within the page of the inner page table

Address-Translation Scheme

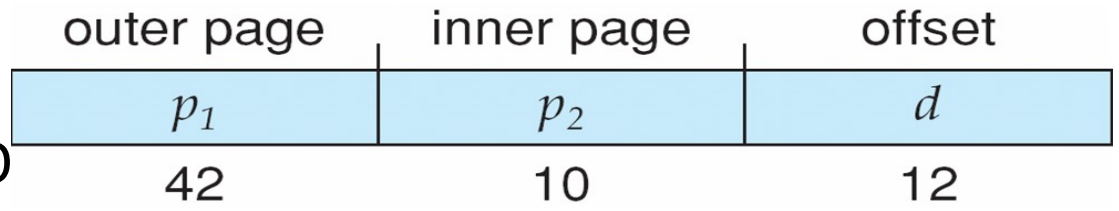
PES UNIVERSITY
CELEBRATING 50 YEARS

- This scheme is known as **forward-mapped page table as** address translation works from the outer page table inward.



64-bit Logical Address Space

- Even two-level paging scheme not sufficient
- If page size is 4 KB (2^{12})
  - Then page table has 2^{52} entries
  - If two level scheme, inner page tables could be 2^{10} 4-byte entries
  - Address would look like

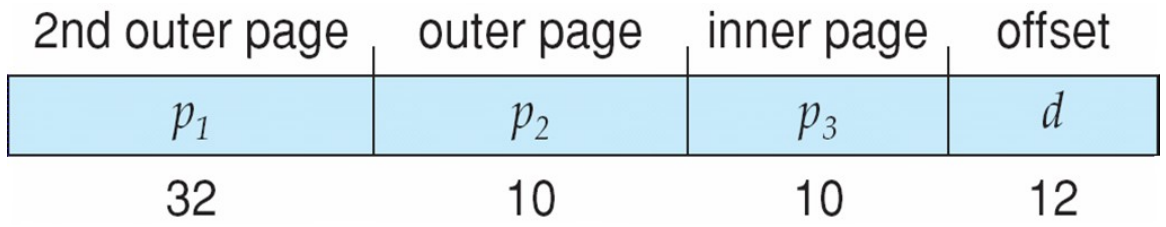


  - O
  - One solution is to add a 2nd outer page table

OPERATING SYSTEMS

Three-level Paging Scheme

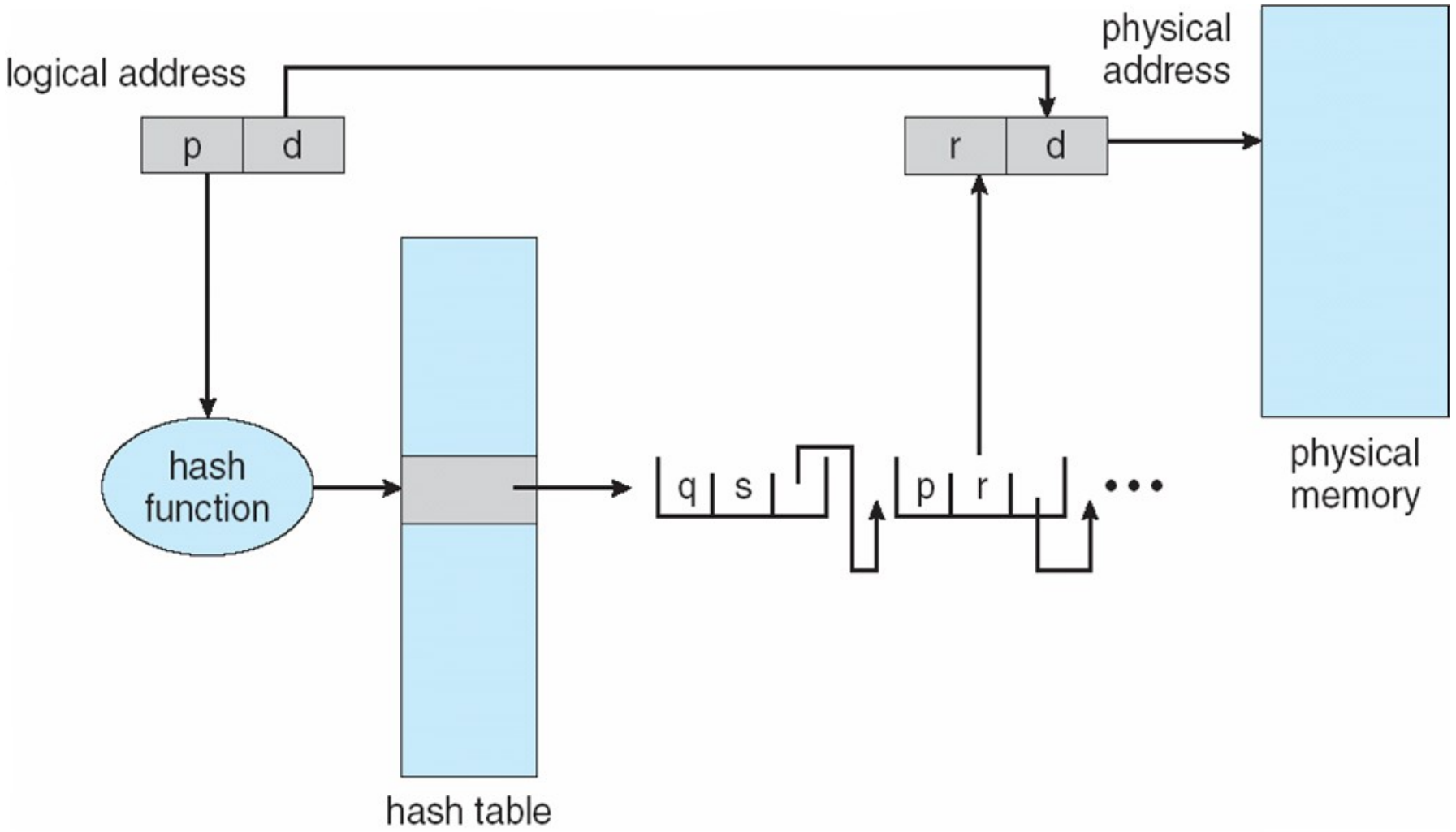
- We can divide the outer page table in various ways.
- For example, we can page the outer page table, giving us a three-level paging scheme. Suppose that the outer page table is made up of standard-size pages (2^{10} entries, or 2^{12} bytes).
- In this case, a 64-bit address space is still daunting:



- The next step would be a four-level paging scheme, where the second-level outer page table itself is also paged, and so forth.

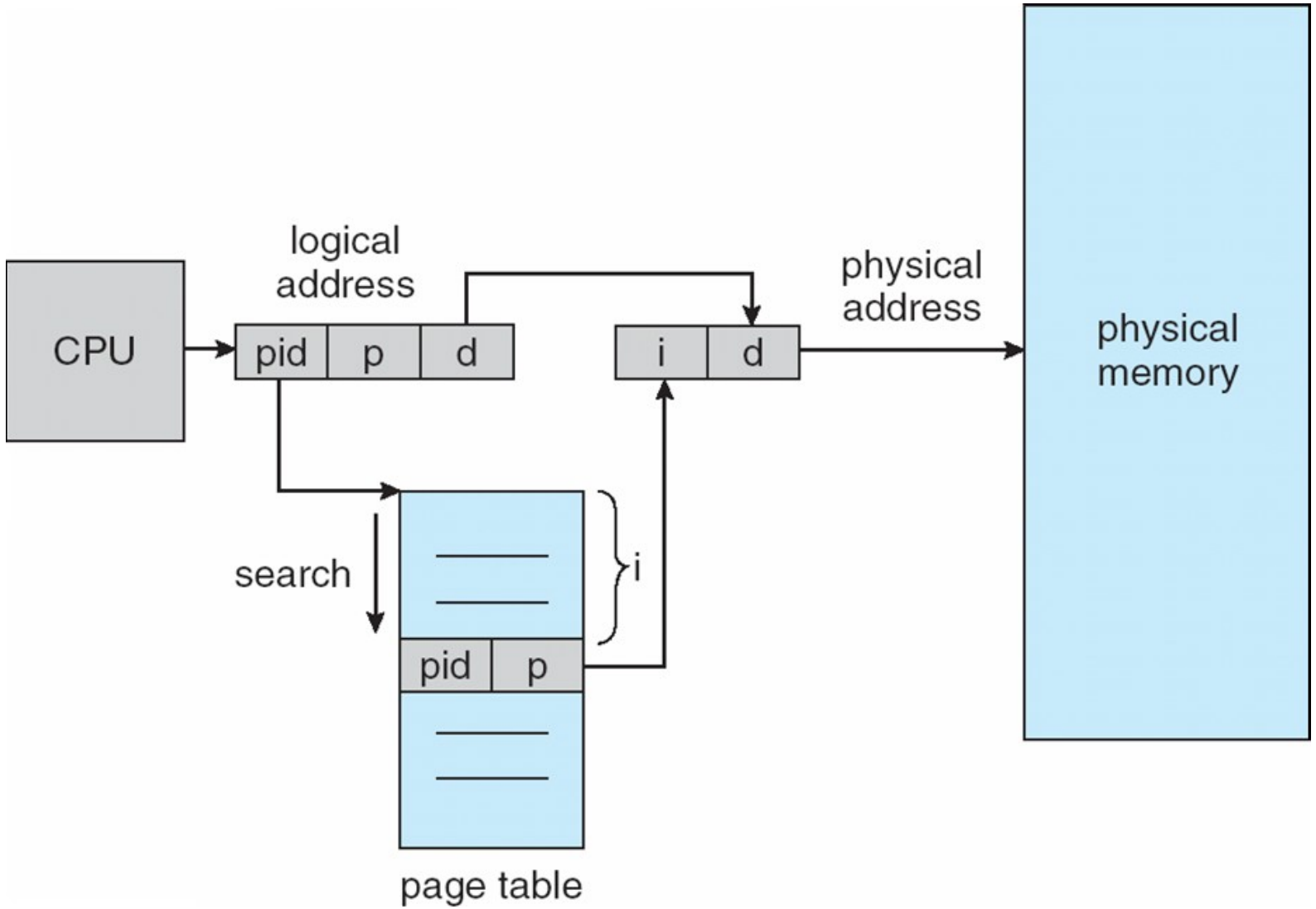
OPERATING SYSTEMS

Hashed Page Table



OPERATING SYSTEMS

Inverted Page Table Architecture



OPERATING SYSTEMS

Inverted Page Table



- Rather than each process having a page table and keeping track of all possible logical pages, track all physical pages
- One entry for each real page of memory
- Entry consists of the virtual address of the page stored in that real memory location, with information about the process that owns that page
- Decreases memory needed to store each page table, but increases time needed to search the table when a page reference occurs
- Use hash table to limit the search to one — or at most a few — page-table entries
  - TLB can accelerate access
- But how to implement shared memory?
  - One mapping of a virtual address to the shared physical address

Inverted Page Table

- Systems that use inverted page tables have difficulty in implementing shared memory.
- Shared memory is usually implemented as multiple virtual addresses are mapped to one physical address.
- There is only one virtual page entry for every physical page
- One physical page cannot have two (or more) shared virtual addresses.
- A simple technique for addressing this issue is to allow the page table to contain only one mapping of a virtual address to the shared physical address.
- references to virtual addresses that are not mapped result in page faults

PES UNIVERSITY
CELEBRATING 50 YEARS

Linux Angle

- How does Linux handle the TLB bottleneck?
- If a database needs 100GB of RAM, the TLB (with only 1024 entries of 4KB) will constantly miss.
- Modern OSes allow processes to request "Huge Pages" (e.g., 2MB or 1GB blocks) so a single TLB entry can cover vastly more memory space.

PES UNIVERSITY
CELEBRATING 50 YEARS

Summary: Segmentation & Paging



Segment Mapping

- **Segment registers** reside directly within the processor for rapid translation.
- **Segment ID** is associated with every memory access.
  - Often extracted from a portion of the virtual address.
  - Can instead be loaded directly from instruction bits (e.g., x86 architecture).
- Each segment contains explicit **base and limit** info, adjusting offsets by adding the base.

Page Tables

- Memory is divided into structured, **fixed-sized chunks** (pages and frames).
- **Virtual page numbers** are systematically mapped to physical page numbers via the page table.
- The **offset** of the virtual address remains identical in the physical address.
- **Large page tables** can be dynamically placed inside virtual memory to save physical RAM.

Multi-Level Tables

- The **virtual address** is mapped through a hierarchical series of nested lookup tables.
- This multi-level structure permits **sparse population** of the process address space.
- Avoids the massive memory overhead of maintaining large, contiguous single-level page tables for unused memory regions.

Questions

- Q1: In a paging system, what is the difference between a Page and a Frame?
- Q2: Why is the page offset (d) not translated by the Memory Management Unit (MMU)?
- Q3: What is the "two-memory-access problem" in a basic paging system?
- Q4: What is the purpose of an ASID (Address-Space Identifier) in a TLB?
- Q5: A system has a TLB hit rate of 90%. It takes 5ns to access the TLB, and 100ns to access main memory. What is the Effective Access Time (EAT)?
- Q6: What specific type of fragmentation does Paging eliminate, and which type does it not?
- Q7: How does a paging architecture implement shared memory (like sharing the C standard library)?
- Q8: Why do modern 64-bit operating systems use hierarchical (multi-level) page tables rather than a single contiguous page table?
- Q10: Why did Inverted Page Tables largely fall out of favour compared to Hierarchical Page Tables in modern computing?

PES UNIVERSITY
CELEBRATING 50 YEARS

OPERATING SYSTEMS

Problems

Q1: Consider a paging system with the page table stored in memory. a. If a memory reference takes 50 nanoseconds, how long does a paged memory reference take? b. If we add TLBs, and 75 percent of all page-table references are found in the TLBs, what is the effective memory reference time? (Assume that finding a page-table entry in the TLBs takes 2 nanoseconds, if the entry is present.)

Given:

Memory access time = **50 ns**

TLB lookup time = **2 ns**

TLB hit ratio = **75% = 0.75**

(a) Paged memory reference (without TLB)

When the page table is stored in memory:

Access the page table in memory = **50 ns**

Access the actual data in memory = **50 ns**

So, Total *memory reference time* $= 50 + 50 = 100\ ns$

(b) Effective memory reference time (with TLB)

There are two cases:

Case 1: TLB Hit (75%) TLB lookup = **2 ns,** Memory access = **50 ns** Total: $2 + 50 = 52\ ns$

Case 2: TLB Miss (25%) TLB lookup = **2 ns,** Page table access = **50 ns** Actual memory access = **50 ns**

Total: $2 + 50 + 50 = 102\ ns$

Effective Access Time (EAT): EAT $= (0.75 \times 52) + (0.25 \times 102) = 39 + 25.5 = 64.5\ ns$

Problems

Q1: Consider a paging system where the page table is stored in main memory.
Given:
A memory access takes **80 nanoseconds**.
A **TLB (Translation Lookaside Buffer)** is added.
The TLB lookup time is **5 nanoseconds**.
90% of page-table references are found in the TLB.
Answer the following:
a. If there is **no TLB**, how long does one paged memory reference take?
b. With the TLB, what is the **effective memory access time (EAT)**?

PES UNIVERSITY
CELEBRATING 50 YEARS

OPERATING SYSTEMS

Problems

Q1: Consider a paging system with the page table stored in memory. a. If a memory reference takes 50 nanoseconds, how long does a paged memory reference take? b. If we add TLBs, and 75 percent of all page-table references are found in the TLBs, what is the effective memory reference time? (Assume that finding a page-table entry in the TLBs takes 2 nanoseconds, if the entry is present.)

Given:

Memory access time = **50 ns**

TLB lookup time = **2 ns**

TLB hit ratio = **75% = 0.75**

(a) Paged memory reference (without TLB)

When the page table is stored in memory:

Access the page table in memory = **50 ns**

Access the actual data in memory = **50 ns**

So, Total *memory reference time* $= 50 + 50 = 100\ ns$

(b) Effective memory reference time (with TLB)

There are two cases:

Case 1: TLB Hit (75%) TLB lookup = **2 ns,** Memory access = **50 ns** Total: $2 + 50 = 52\ ns$

Case 2: TLB Miss (25%) TLB lookup = **2 ns,** Page table access = **50 ns** Actual memory access = **50 ns**

Total: $2 + 50 + 50 = 102\ ns$

Effective Access Time (EAT): EAT $= (0.75 \times 52) + (0.25 \times 102) = 39 + 25.5 = 64.5\ ns$

OPERATING SYSTEMS

Problems

Q1: Consider a logical address space of 64 pages of 1,024 words each, mapped onto a physical memory of 32 frames.

a. How many bits are there in the logical address?

b. How many bits are there in the physical address?

a. Number of bits in the logical address

A logical address consists of:

Page number

64 pages = 2^6pages, Needs **6 bits**

Offset within a page

1,024 words per page = 2^{10}words, Needs **10 bits**, No of bits in Logical address=10+6 =16

b. Number of bits in the physical address

A physical address consists of:

Frame number

32 frames = 2^5frames, Needs **5 bits**

Offset within a frame

Frame size = page size = 1,024 words = 2^{10}, Needs **10 bits** Physical address bits $= 5 + 10 = 15$ bits

Q2: A logical address space consists of 128 pages, each containing 512 bytes. The physical memory consists of 64 frames.

Questions

How many bits are required for the logical address?

How many bits are required for the physical address?

How many bits are used for the page number?

How many bits are used for the frame number?

Problems



Q3: A logical address space consists of 128 pages, each containing 512 bytes. The physical memory consists of 64 frames. How many bits are required for the logical address?
How many bits are required for the physical address?
How many bits are used for the page number?
How many bits are used for the frame number?

Pages = 128 = 2^7
Page size = 512 bytes = 2^9
Frames = 64 = 2^6
Logical address:
Page number = 7 bits
Offset = 9 bits
Logical address = 7 + 9 = 16 bits
Physical address:
Frame number = 6 bits
Offset = 9 bits
Physical address = 6 + 9 = 15 bits

Q4: A logical address is 18 bits long. The page size is 1024 bytes. Find: Number of pages, Offset bits, and Page number bits

Solution

Page size: 1024 bytes = 2^{10} , Offset bits = **10**
Logical address = 18 bits
Page number bits= 18 − 10= **8 bits**
Pages= 2^8 = **256 pages**

Examples



Consider that the size of main memory (physical memory) is 512 MB and frame size of 4KB. Calculate(Assume logical address space 2GB)

a.Page offset(d)

b.Bits to address page number/Frame number

c.What will be the total number of frames

d.Number of level of paging

Page table example

Consider a computer system with a 32-bit logical address and 4-KB page size. The system supports up to 512 MB of physical memory. How many entries are there in each of the following?

a. A conventional single-level page table

b. An inverted page table

🔢 Given Data:

- **Logical Address Space:** 32-bit address = 2^{32} bytes
- **Page Size:** 4 KB = 2^{12} bytes
- **Physical Memory Size:** 512 MB = 2^{29} bytes

A a. Conventional Single-Level Page Table:

A conventional page table has one entry per page in the logical address space.

1. **Number of Pages in Logical Address Space:**

$$\text{Number of Pages} = \frac{\text{Logical Address Space}}{\text{Page Size}} = \frac{2^{32}}{2^{12}} = 2^{20} = 1,048,576 \text{ pages}$$

2. **Entries in the Page Table:** Since each page needs an entry, the number of entries in the page table is:

$$2^{20} = 1,048,576 \text{ entries}$$

B b. Inverted Page Table:

An inverted page table has one entry for each physical frame instead of each logical page.

1. **Number of Frames in Physical Memory:**

$$\text{Number of Frames} = \frac{\text{Physical Memory Size}}{\text{Page Size}} = \frac{2^{29}}{2^{12}} = 2^{17} = 131,072 \text{ frames}$$

2. **Entries in the Inverted Page Table:** Since each frame corresponds to an entry, the inverted page table requires:

$$2^{17} = 131,072 \text{ entries}$$

Summary:

Conventional Single-Level Page Table: 1,048,576 entries

Inverted Page Table: 131,072 entries



THANK YOU

Pavan A C

Department of Computer Science Engineering

pavanac@pes.edu